

Merge-Search Diagnostics

Revision: 1 Last modified: 2026-06-06T00:00:00Z

This document explains how the merge service reports per-tracker failures so operators can distinguish “nothing wrong, niche board has no hits for this query” from “upstream is dead” from “plugin is crashing.”

Context: prior to 2026-04-23 a search for `linux` returned 137 total hits and 37 of 40 trackers returned zero with **no explanation**. The root cause was a monkeypatch targeting the wrong `novaprinter` module (see commit `e8b904d`), but the deeper problem was visibility: `TrackerSearchStat.error` was always `None` on empty trackers, so operators couldn’t tell whether an empty chip meant “nothing to see” or “broken.” This doc describes the diagnostic pipeline that replaces that opacity.

Data flow

```
plugin subprocess stderr → _classify_plugin_stderr()
                             →
                             → _last_public_tracker_diag[tracker]
                             →
                             → _search_one() pops + writes to
                             →
                             → TrackerSearchStat.{error,error_type,notes}
                             →
                             → API response `tracker_stats[]` + SSE `tracker_completed`
                             →
                             dashboard chip badge + tracker-stat-dialog "Error" section
```

Every public-tracker subprocess runs in isolation in a Python 3.12 process spawned by `_search_public_tracker` (`download-proxy/src/merge_service/search.py`). Whatever the plugin writes to `stderr` is captured after the subprocess exits (or is killed at the deadline) and passed through `_classify_plugin_stderr` to produce a structured diagnostic.

Subprocess lifecycle

1. **Spawn** “`asyncio.create_subprocess_exec(..., start_new_session=True)`” gives the plugin its own process group so any threads or forked workers can be terminated together.
2. **Stream** “`stdout` is read line-by-line with a per-read timeout. Each NDJSON row is parsed as it arrives; partial results are preserved even if the deadline fires mid-scrape.
3. **Deadline** “when `PUBLIC_TRACKER_DEADLINE_SECONDS` is reached the loop breaks with `killed_by_deadline=True`.”

4. **Kill** â€” `proc.kill()` (SIGKILL to the direct child) followed by `os.killpg(os.getpgid(proc.pid), signal.SIGKILL)` (SIGKILL to the entire process group). This catches plugins that spawn child processes or background threads.
5. **Wait** â€” `asyncio.wait_for(proc.wait(), timeout=5.0)` waits up to 5 seconds for the process to actually die. If it doesnâ€™t, we log a warning and abandon the zombie â€” the orchestrator must NEVER hang waiting for a defunct subprocess.
6. **Stderr** â€” `asyncio.wait_for(proc.stderr.read(), timeout=5.0)` reads stderr with the same 5-second cap. If the pipe is still open (zombie), we abandon it and move on.
7. **Backstop** â€” `_search_tracker` wraps the entire `_search_public_tracker` call in an outer `asyncio.wait_for(..., deadline+10)` so even if every internal cleanup step has a bug, the coroutine always returns.
8. **Gather defence** â€” `_run_search` uses `asyncio.gather(..., return_exceptions=True)` so one tracker failing (or hanging) cannot prevent the rest of the fan-out from completing.

Error classes (error_type)

error_type	Meaning	Operator action
upstream_http_403	Remote tracker returned 403 Forbidden	Likely geoblock or CDN WAF. Try VPN.
upstream_http_404	Remote tracker returned 404 Not Found	Domain moved â€” update PUBLIC_TRACKERS URL
upstream_timeout	Remote gateway timed out	Transient â€” retry or bump PUBLIC_TRACKER_DEADLINE_SECONDS
dns_failure	Tracker hostname does not resolve	Domain is dead â€” move to DEAD_PUBLIC_TRACKERS
tls_failure	TLS handshake with upstream failed	Cert issue upstream. No action.
upstream_incomplete	Upstream closed connection mid-response	Transient â€” retry
plugin_env_missing	Plugin needs a file/dir not present in the container	Check plugin source; add dir to Dockerfile
plugin_parse_failure	Plugin regex/JSON parse failed (upstream HTML rotated)	Patch plugin regex
plugin_crashed	Plugin raised an uncategoryed exception	Read <code>notes.stderr_tail</code> for traceback
deadline_timeout		Bump deadline or split the work

<code>error_type</code>	Meaning	Operator action
	Plugin was still running after per-tracker deadline	
<code>auth_failure</code>	Private tracker login returned no session cookie	Check credentials in <code>.env</code> ; may be transient CAPTCHA
<code>upstream_captcha</code>	Private tracker served a CAPTCHA page instead of results	Wait a few minutes and retry; rotate IP if persistent

When the plugin runs cleanly but doesn't emit any rows, `error_type` is `None` and `status` stays at empty. This is the "genuinely no hits for this query on this board" path e.g. searching `linux` on an anime-only tracker.

Environment variables

`PUBLIC_TRACKER_DEADLINE_SECONDS`

Default: 60. Clamped to `[5, 120]`.

How long each public-tracker subprocess may run before it is SIGKILLed. Because `_search_public_tracker` streams results as NDJSON with `sys.stdout.flush()`, partial results are preserved even if the deadline fires mid-scrape. When that happens, `TrackerSearchStat.notes["deadline_hit"] = True` and the dashboard chip shows a stopwatch icon.

Raise this if you have slow upstreams and are willing to wait " but remember that the orchestrator runs up to `MAX_CONCURRENT_TRACKERS` fan-out tasks at a time, so wall-clock latency grows with deadline $\sim \text{ceil}(\text{trackers} / \text{max_concurrent})$.

`ENABLE_DEAD_TRACKERS`

Default: 1 (all trackers exposed; dead ones filtered by `DEAD_PUBLIC_TRACKERS`).

Public trackers that are empirically known-dead as of 2026-04-23 live in `DEAD_PUBLIC_TRACKERS` in `search.py` and are excluded from the fan-out. Set `ENABLE_DEAD_TRACKERS=0` to hide them. Set `ENABLE_DEAD_TRACKERS=1` (default) to include them for testing whether an upstream has recovered, or when operating through a proxy/VPN that bypasses geoblocks.

The known-dead list:

- HTTP 403 (Cloudflare/geoblock): `eztv`, `bt4g`, `extratorrent`, `one337x`, `bitru`
- HTTP 404 (site down): `ali213`
- Gateway timeout: `audiobookbay`
- DNS failure (domain dead): `pctorrent`, `yihua`
- TLS failure: `xfsub`
- Plugin crash (site down): `none`
- API changed: `anilibra` (returns 400 Unknown query)

- Site rebrand/redesign: `solidtorrents` (→ `bitsearch.to`), `therarbg` (new HTML), `gamestorrents` (WordPress redesign), `btsow` (JS redirect challenge)
- Upstream dead: `torrentfunk` (HTTP 500)

These plugins are still installed and the classifier still reports their real reasons when `ENABLE_DEAD_TRACKERS=1`, so it's easy to spot when an upstream starts working again and move the entry out of the set.

MAX_CONCURRENT_TRACKERS

Default: 5. Semaphore-bounded fan-out width. Wall-clock latency is approximately $\text{deadline} \sim (\text{tracker_count} / \text{max_concurrent})$ in the worst case. Lower this on constrained hosts to keep memory bounded.

MAX_CONCURRENT_SEARCHES

Default: 8. Per-orchestrator cap on the number of in-flight `_run_search` fan-outs. When saturated, both `POST /api/v1/search` and `POST /api/v1/search/sync` return HTTP 429 with a `retry_shortly` hint so callers back off rather than piling more subprocess work on a starving event loop.

Stress tests revealed that without this cap, 50 rapid POSTs would spawn $50 \times 24 = 1200$ simultaneous subprocess spawns and even `/health` stopped responding. With the cap in place the service remains responsive (at most $8 \times 24 = \sim 200$ subprocesses mid-flight, plus the 5-per-fan-out semaphore actually runs closer to 40).

Dashboard chip legend

Icon	Meaning
✓ N	Success ✓ N results
^ ...	Empty ✓ plugin ran cleanly, found nothing for this query
✗	Error ✓ hover for <code>error_type</code> , click for full dialog
⚡ ±	Deadline hit ✓ results are truncated to whatever flushed
🔑	Authenticated session used

Click any chip to open the **tracker-stat dialog** which shows the full `TrackerSearchStat` including `notes.stderr_tail` for crashed plugins.

Rebuild / restart contract

`docker-compose.yml` bind-mounts `./download-proxy/` into the container at `/config/download-proxy`. That means:

- **Python source changes** in `download-proxy/src/` → `podman restart qbittorrent-proxy` picks them up; no rebuild needed.
- **Plugin changes** in `plugins/` → `./install-plugin.sh` copies them into `./config/qBittorrent/nova3/engines/`, which is also bind-mounted; restart again picks them up.
- `docker-compose.yml / start-proxy.sh / env changes` → `podman compose down && podman compose up -d`.

Full rebuild (`podman build`) is only necessary when the base image (`python:3.12-alpine`) itself needs to change.

Troubleshooting

All public trackers return 0. The subprocess capture is broken. First thing to check: does the script in `_search_public_tracker` still do `import novaprinter as _np`? If it says `engines.novaprinter`, youâ€™ve reintroduced the 37-empty-trackers bug. See commit `e8b904d`.

One tracker goes to 0 intermittently. Look at `tracker_stats[] .error`. If itâ€™s consistently `upstream_http_403`, the geoblock probably caught you; if it bounces between success and `upstream_timeout`, the upstream is rate-limiting.

results_count is capped at some suspicious round number. Check `notes.deadline_hit`. If true, raise `PUBLIC_TRACKER_DEADLINE_SECONDS`.

Search hangs forever â€” one tracker is stuck in cleanup. Prior to 2026-04-24 a plugin in an uninterruptible sleep state could deadlock the entire orchestrator because `stderr.read()` was called before `proc.wait()`. The fix reordered cleanup, added process-group kills, and wrapped every blocking call in a 5-second timeout. If you still see hangs, check the logs for the warning "proc.wait() timed out during cleanup â€” abandoning zombie".

I added a tracker but it never appears in the fan-out. Itâ€™s probably in `DEAD_PUBLIC_TRACKERS`. Move it out, or set `ENABLE_DEAD_TRACKERS=1` temporarily.